Post-Quantum Cryptography for Blockchain

Student Lab Instructions

1. Overview

In this lab you will implement a Learning With Errors (LWE) encryption scheme from scratch using only basic Python. You will then use it to encrypt a simulated blockchain transaction. LWE is the mathematical foundation of NIST's post-quantum cryptography standards (ML-KEM / Kyber, ML-DSA / Dilithium).

The lab notebook (pqc_blockchain_lab.ipynb) contains sections with background reading, code cells with TODO tasks to complete, and written-answer questions. All code uses only Python standard library (no NumPy, no external packages).

2. Setup

You need Python 3.8+ with Jupyter Notebook or Jupyter Lab.

If you need to install Jupyter:

```
pip install jupyter
jupyter notebook pqc_blockchain_lab.ipynb
```

3. Notebook Structure

The notebook has 8 parts. Complete them in order. Each builds on the previous.

Part 1 -- Reading + Question 1
The quantum threat to blockchain. Read, run the scaling demo, answer Q1.

Part 2 -- Reading
Lattice-based cryptography overview. Just read.

Part 3 -- Tasks 3a to 3f (6 helper functions)
3a: dot(a, b) -- dot product of two vectors
3b: mat_vec_mul(A, v, q) -- matrix-vector multiply (mod q)
3c: small_vector(n) -- random vector with entries -1, 0, 1
3d: random_vector(n, q) -- uniform random vector
3e: transpose(A) -- matrix transpose
3f: random_matrix(m, n, q) -- random matrix
Each has a test cell. Make sure it passes before continuing.

Part 4 -- Task 4 + Question 2
Implement LWE key generation: (A, b = A*s + e), s key pair.
Then answer Q2: why does noise make LWE hard?

Part 5 -- Task 5
Implement LWE encryption for a single bit: (u, v) = (A^T*r, b*r + m*q/2).

Part 6 -- Task 6 + Question 3
Implement LWE decryption. Compute d = (v - s*u) % q, decode the bit.
Run full test (7 bits). All must pass. Then answer Q3.

Part 7 -- Tasks 7a, 7b

7a: message_to_bits() and bits_to_message() -- string/bit conversion.
7b: Encrypt a JSON transaction string bit-by-bit with LWE, decrypt, verify.

Part 8 -- Reading + Question 4

Current NIST/Google PQC standards. Answer Q4.

4. Coding Tips

- Use loops and list comprehensions -- no NumPy or external libraries
- Each function has an inline test cell; run it immediately to verify
- Global variables n=8, q=97 are set in Part 4 and reused later
- If a test fails, read the error message and check your logic
- All mod operations use Python's % operator (always non-negative)
- Error entries are in {-1, 0, 1}, not just {0, 1}

5. Written Questions

Answer all four questions in the notebook's markdown cells (2-4 sentences each, in your own words):

Written Question Q1

Why does the quantum speedup matter for blockchain security?
What happens to ECDSA if Shor's algorithm runs on a large enough quantum computer?

Written Question Q2

Why does adding small noise e to the public key make the LWE problem hard?
What would happen if we set e = 0 (all zeros)?

Written Question Q3

Why can't an attacker who sees the public key (A, b) and the ciphertext (u, v)
recover the message, while the legitimate user (who knows s) can?
Use the math from the decryption derivation in your explanation.

Written Question Q4

Why is lattice-based cryptography a good replacement for ECC in blockchain?
What advantages does it have over other post-quantum approaches like hash-based
or code-based cryptography?

6. Submission Checklist

- [ ] All 11 code tasks (3a-3f, 4, 5, 6, 7a, 7b) are complete and pass their tests
- [ ] The full test in Part 6 shows 'OK' for all 7 bits
- [ ] The blockchain transaction in Part 7b prints 'Match: True'
- [ ] All 4 written questions are answered in the markdown cells
- [ ] Notebook is saved as pqc_blockchain_lab.ipynb with your answers

7. Quick Reference: LWE Math

Key Generation

```
Public key: (A, b)  where b = A*s + e (mod q)
Secret key: s
```

Encryption (bit m in {0,1})

```
u = A^T * r            (mod q)
v = b . r + m * q//2  (mod q)
Ciphertext: (u, v)
```

Decryption

```
d = v - s . u  (mod q)
  = e . r + m * q//2
If d > q/4 and d < 3*q/4 -> bit = 1
Else                    -> bit = 0
```

8. References

- NIST PQC Project: https://csrc.nist.gov/projects/post-quantum-cryptography
- FIPS 203 (ML-KEM): https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf
- FIPS 204 (ML-DSA): https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf
- FIPS 205 (SLH-DSA): https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf

- Google PQC Migration: https://security.googleblog.com/2023/08/transitioning-to-post-quantum.html
- Regev, O. (2005). 'On lattices, learning with errors...'
- Shor, P. (1994). 'Algorithms for quantum computation...'